XMOS Controller Area Network Protocol App Note

(VERSION 1.0)



2009/03/16

Authors:

SULEIMAN ABU KHARMEH

Copyright © 2009, XMOS Ltd.
All Rights Reserved

1 Introduction

This document describes the XMOS implementation of the CAN protocol on an XS1-G4 chip using the XC1 development board.

Please note this is an early release of this API as the implementation is not complete yet. This document will be updated to reflect further progress.

2 API details

Details of the API are included in files:

- include/CanConfig.h
- include/CanFunctions.h
- include/CanIncludes.h

Include the **include/CanIncludes.h** file which brings in all the needed APIs to your project.

This API implements the bottom two layers of the ISO/OSI reference model, namely the Physical Layer and the Data Link Layer. The implementation maps those two layers into two separate hardware threads:

- void **CanPhyRx** (chanend toData, buffered in port:32 canRX, port canTX)
- void **CanDataRx** (chanend toNetwork, chanend fromPhy)
- void **CanPhyTx** (chanend fromData, in port canRX, port canTX)
- void **CanDataTx** (chanend fromNetwork, chanend toPhy)

For simplicity, the transmit and receive functions are implemented using separate threads. In the next phase of the project, those should be merged into the same thread. This is due to the fact that a transmitter should switch to being a receiver when it loses arbitration.

The CanPhy(Rx/Tx) threads take the CAN transceiver TXD and RXD ports. Also they take a channel end that connects them to the CanData(Rx/Tx) threads which implement the Data Link Layer. The CanData(Rx/Tx) threads take two channel ends as parameters, one that connects them to the CanPhy(Rx/Tx) threads, and the other connects them to the user code, in this case the network layer where a simple loop back function is implemented.

The API will deliver parsed CAN packets through the toNetwork channel end to the developer. The format of the packet is as follows:

struct canPacket

unsigned SOF;
unsigned ID;
unsigned SRR;
unsigned IDE;
unsigned EID;
unsigned RTR;
unsigned RB1;
unsigned RB0;
unsigned DLC;
unsigned DATA[8];
unsigned CRC;
unsigned CRCdel;
unsigned ACKdel;
unsigned _EOF;

;

All the fields are 32 bit unsigned values for now.

Please refer to the CAN specification for more details on the CAN packet format[2].

At the time of this writing basic receiver and transmitter functions were implemented with no error checking/handling, or arbitration.

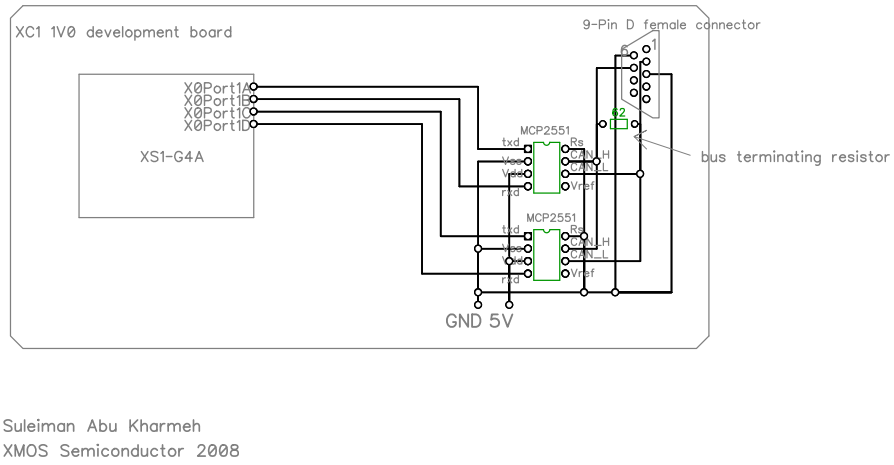


Figure 1 *Development platform: XC1 board with MCP2551 transceiver*

3 Testbench Setup

For development purposes, a CAN-to-USB dongle[1] was used to connect to the XC1 development board[3]. The C# application provided was used to send and receive CAN packets to the development board. Please see figure 1 for more details.

At this stage of the implementation the application can be built only to output debugging information related to the Physical layer, or the Data Link Layer.

In the first case **#define CAN_DEBUG1** the application will output (through the debugging interface) all the bit values captured and their times as captured by the port. See figure 2 for more details. This is done after de-stuffing, so it can be noticed clearly from the log where stuffed bits have been removed.

In the second case **#define CAN_DEBUG2** where the Data Link Layer is being debugged, the application will output all captured packets after parsing into the canPacket structure above including all the data bytes if present. See figure 3 for more details.

For testing purposes the kit can act as a transmitter, receiver, or both at the same time. To switch the tramsmitter stack on you declare **#define CAN_TX**, and to switch the to switch the receiver stack on you declare **#define CAN_RX**. Hence you can have the following different setup senarios:

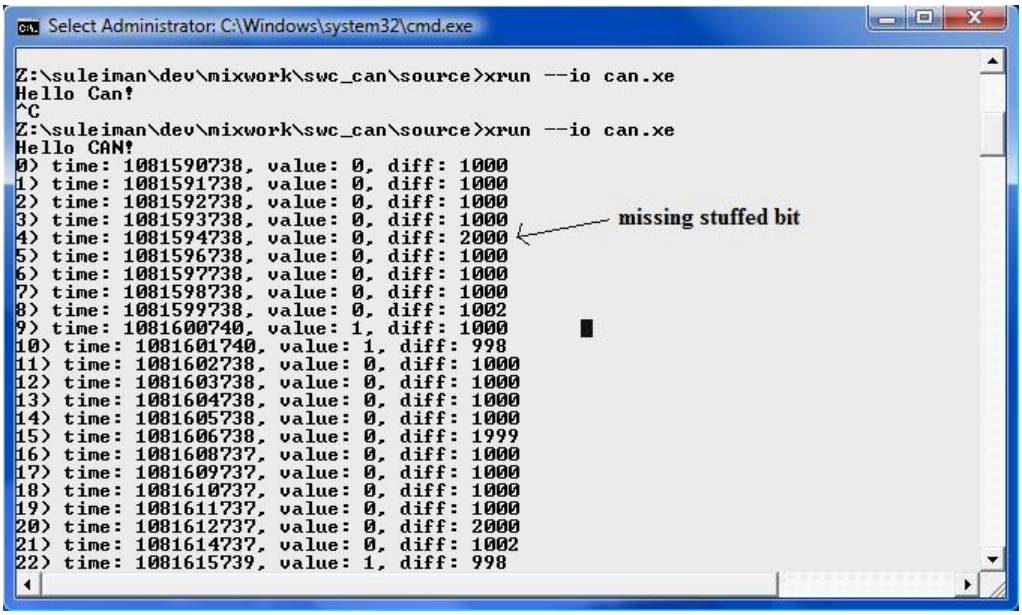


Figure 2 *Physical Link layer debugging (CAN_DEBUG1 output)*

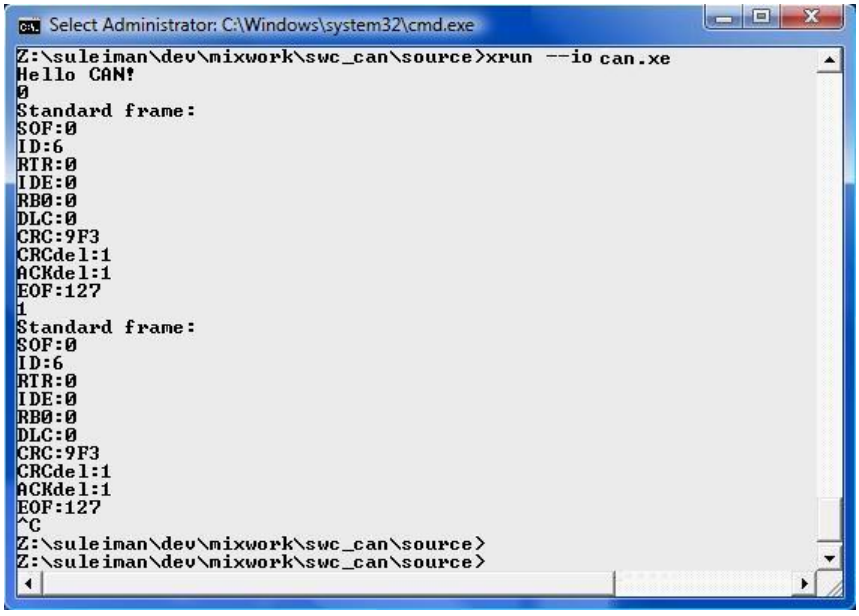


Figure 3 *Data Link layer debugging (CAN_DEBUG2 output)*

- Switch both the transmitter stack and the receiver stack on, in which case the kit would would behave in loopback mode and would not need to interact with the PC side.
- Switch only the transmitter stack on, and connect to the PC application through the CAN-to-USB dongle, and use one of the example projects provided with the CAN-to-USB dongle driver to receive packets from the kit.
- Switch only the receiver stack on, and connect to the PC application through the CAN-to-USB dongle, and use one of the example projects provided with the CAN-to-USB dongle driver to send packets to the kit.

In all the cases above, you should observe the debugging information displayed according to the debugging level configured. Also the PC application should provide good insight on the activity of the CAN-to-USB dongle.

To build the project from the command line, go into the `build` directory and type `make -f ../source/Makefile`, this should produce the `can.xe` executable file. Run the `can.xe` executable on the XMOS development kit using the command `xrun --io can.xe`

4 Timing and Synchronisation

In this section, an explanation of how the XS1 chip synchronises to the CAN bit stream. The reader is assumed to be familiar with the details involved in the CAN protocol synchronisation. For further information, please consult the CAN ISO specification[2].

The bit time is devided into 25 quantas. Based on a 100MHz port clock, this devides into 4 port clocks per CAN time quanta. This should scale down easily when slower port clocks are considered.

The bit time is divided as follows:

Function	Time Quantas
syncronisation	1
propagation	8
phase buffer 1	8
phase buffer 2	8

On the XS1 chip, a 1 bit port is attached to the CAN RX pin on the CAN transceiver. This port is configured in 32 bit serialised input mode. This means that at each port clock (which corresponds to a CAN time quanta) the port will sample the CAN bus, and store the sample in its shift register for the software to process later.

The software would do a hard synchronisation on the start of the frame, and then it would do a repititive timed input, those inputs are timed past the propagation delay of the next bit. This means that the inputted 32 bit value looks as follows:

xxx	N Phase_Seg1	N Phase_Seg2	N+1 Sync_Seg	N+1 Prop_Seg

Hence from each input, the sample of the current bit (which is located between Phase_Seg1 and Phase_Seg2) is captured and passed to the data layer, and also the synchronisation edge of the next bit is located. The synchronisation edge can be located anywhere within the Phase_Seg2 of the previous bit (bit N in the figure above)-i.e. early edge- and Prop_Seg of the next bit (bit N+1 in the figure above) -i.e. late edge-. Based on the location of the synchronisation edge within the permitted range, and the rules that govern the amount of phase shift allowed, the time of the next timed input is calculated, and the port is setup to perform the next input accordingly.

5 Results

At the time of this writing, both the receiver and transmitter interface were implemented at the physical layer level. Basic support for standard and extended packets is provided, and the application was tested for 100Kbps, 500Kbps and 1000Kbps. No support for error conditions/packets is implemented yet, including the generation/checking of the CRC.

6 Summary

The XMOS devices are ideal for protocols which map well into the ISO/OSI reference model, through the mapping of different layers of the protocol into different communicating hardware threads. Controller Area Network is such a protocol

where each layer of the ISO/OSI stack can be implemented using one thread. More processing power can be achieved if the receive and transmit interface could be decoupled into separate thread stack.

In this application note, we document the XMOS implementation of the CAN protocol interface using “XC”, the high level Software Defined Silicon programming language.

This project will shortly be available on xlinkers.org, XMOS’s community website.

References

[1] LAWICEL AB. Canusb - manual. online http://www.canusb.com/documents/canusb_manual.pdf, February 2006.

[2] International Organization for Standardization ISO. Road vehicles – controller area network (can) – part 1: Data link layer and physical signalling. ISO copyright office, Switzerland, December 2003.

[3] XMOS Semiconductor. Xc-1 development kit quick start guide. online https://www.xmos.com/system/files/xc1-quick-start-guide.pdf, December 2008.

XMOS Ltd is the owner or licensee of this design, code, or Information (collectively, the “Information”) and is providing it to you “AS IS” with no warranty of any kind, express or implied and shall have no liability in relation to its use. XMOS Ltd makes no representation that the Information, or any particular implementation thereof, is or will be free from any claims of infringement and again, shall have no liability in relation to any such claims.

(c) 2009 XMOS Limited - All Rights Reserved